UPEC
Connaissance - Action

UNIVERSITÉ
PARIS-EST CRÉTEIL
VAL DE MARNE

20/04/2025

RAPPORT SAE 2.1

MEMBRES :

BAMBA Wede Espera
CHACHOUA Shirel

TABLE DES MATIÈRES

I.Introduction……p.2

II.Description……p.2
1. Fonctionnalités……p.2
2. Structure du Programme……..p.3
   a) Classe Grille……p.5
   b) Classe Bloc……..p.6
   c) Classe Score……p.7
   d) Classes Jeu et Main……...p.7
3. Algorithme des blocs. ……..p.8

III.Conclusion……..……p.9
1. Conclusion Wede……..p.9
2. Conclusion Shirel……..…p.10

I. Introduction

Dans le cadre de la SAE 2.1, nous avons développé un jeu contenant une grille composée de différents blocs de couleurs.
Le concept du jeu était de permettre au joueur de générer une grille aléatoire ou à partir d'un fichier, et de supprimer les blocs de couleurs tout en visualisant son score. Le jeu prend fin lorsque la grille est vide.

Le langage utilisé lors du développement du jeu est le langage Java,
L'éditeur de texte utilisé est Sublime Text.

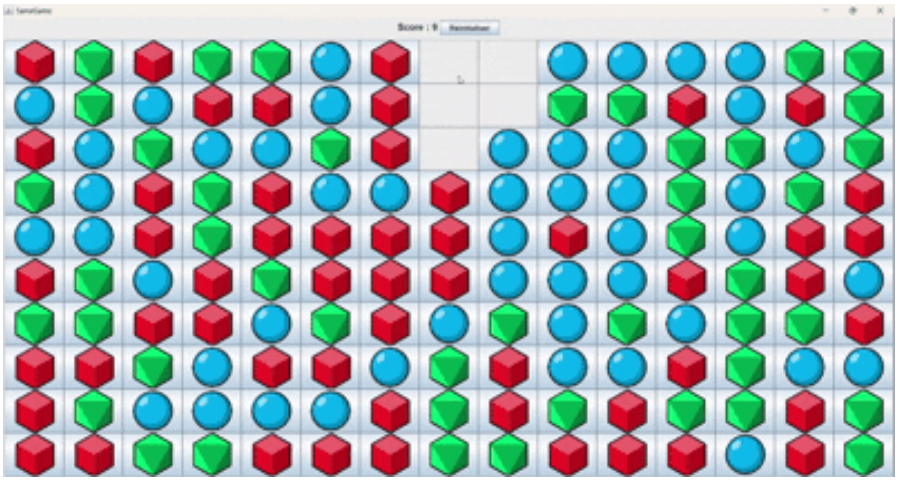
Le serveur GIT nous a permis de collaborer et d'échanger nos codes.

II. Description

1. Fonctionnalités du Jeu

Le jeu génère une fenêtre composée d'une grille d'éléments.
Les éléments sont des images représentant trois formes de couleurs différentes : Rouge, Verte, Bleue.

Lorsque le joueur lance le jeu et survole les différents blocs de formes avec la souris, un surlignement jaune apparait (voir vidéo ci-dessous).

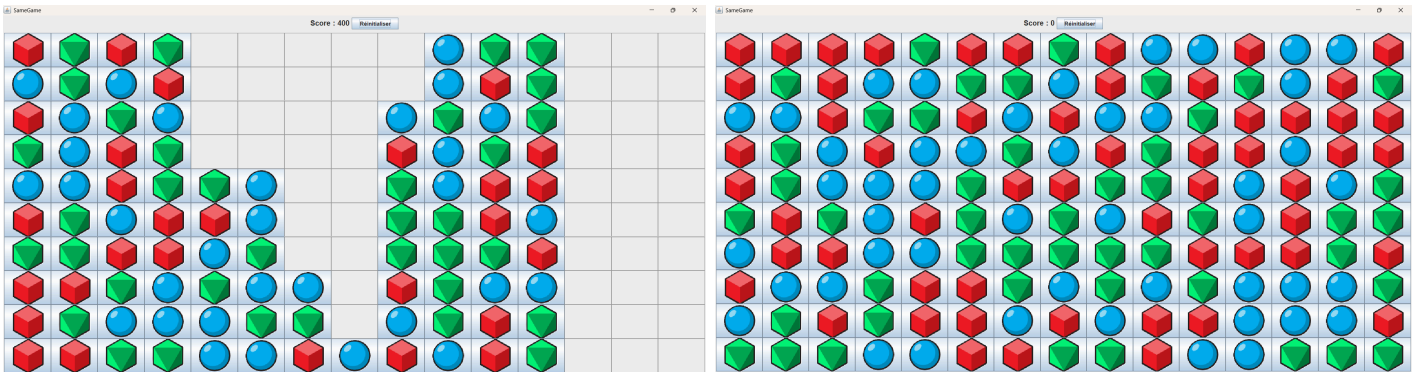


Capture du fonctionnement du jeu

Lorsque le joueur clique sur un bloc, celui-ci disparaît et les éléments restants sont repositionnés.
Si une colonne est vide, les colonnes restantes se décalent vers la gauche.

Aussi, le score du joueur est affiché en haut de l'écran et est mis à jour lorsqu'un bloc est sélectionné.

Le bouton *Réinitialiser*, également situé en haut de l'écran permet d'afficher une grille vide et de remettre le score à zéro.



Capture de la grille Avant/Après réinitialisation

2. Structure du Programme

Le programme regroupe 5 fichiers :

- Grille.java
- Bloc.java
- Score.java
- Jeu.java
- Main.java

Chaque fichier est consacré à l'une des cinq classes principales de notre jeu. Ces classes ont été déterminées via un travail de réflexion en amont, et recensés dans un diagramme de classes à partir du logiciel StarUML :

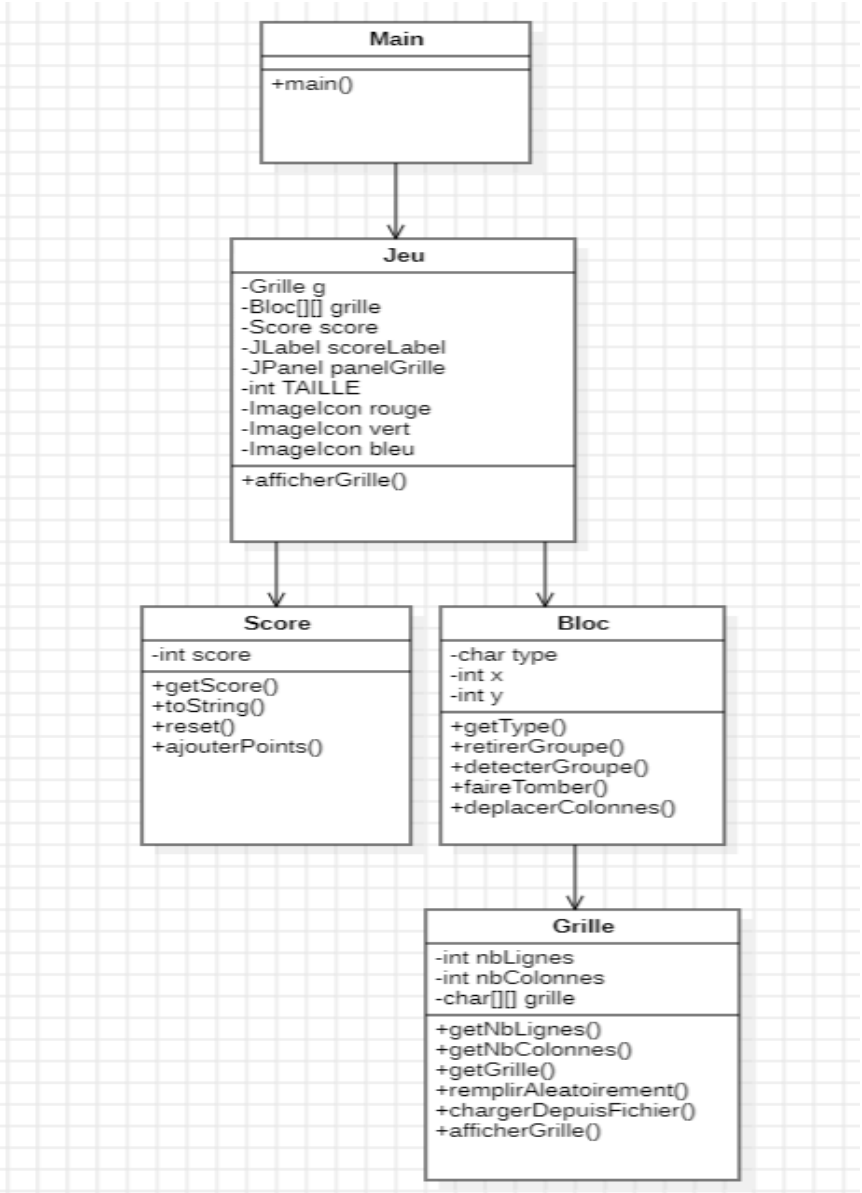


Diagramme de classes

La classe Jeu nécessite la création des classes Score et Bloc (elle même reliée à la classe Grille), puisque ces dernières gèrent la logique du jeu tandis que la classe Jeu gère l'interface graphique.
La classe Main est la dernière classe créée, elle est uniquement responsable du lancement du jeu.

a) Classe Grille

La classe Grille permet de former une grille de dimension 10x15 contenant différents éléments : les caractères R,V,B (qui seront plus tard reliés à leurs images correspondantes).
Les éléments de la grille peuvent être générés aléatoirement ou à partir d'un fichier.

Ainsi, la classe contient les méthodes :

- remplirAleatoirement()

  A l'aide de la classe Random, le programme remplit aléatoirement la grille des éléments R,V,B (instanciés au préalable dans une liste de chaînes de caractères).
  Une boucle a été mise en place pour s'assurer que les dimensions de la grille sont respectées, et donc qu'un élément n'est pas ajouté en dehors des limites.

- chargerDepuisFichier(), selectionnerFichier()

  La méthode chargerDepuisFichier prend un fichier en argument et récupère les caractères du fichier. Elle s'assure que le nombre de caractères du fichier est compatible avec les dimensions de la grille. Le fichier doit donc respecter un certain format (avec le bon nombre de lignes et de colonnes).

  La méthode selectionneFichier utilise la méthode FileChooser pour permettre à l'utilisateur de sélectionner un fichier dans l'explorateur de fichiers. Elle présente uniquement les fichiers .txt disponibles sur l'ordinateur.

- afficherGrille()

  Cette méthode a été créée dans le but de tester la classe en affichant les résultats des méthodes conçues ci-dessus.

b) Classe Bloc

La classe Bloc analyse les éléments de la grille pour former des blocs (regroupant les éléments de même type et adjacents).
Elle gère les méthodes principales de la logique du jeu. On retrouve 4 méthodes :

- detecterGroupe()

Cette méthode parcourt toute la grille pour identifier des groupes de blocs adjacents ayant la même couleur. Seuls les groupes contenant au moins deux blocs sont conservés. Une fonction récursive et une double boucle for ont été utilisées, permettant de renvoyer une liste de groupes de blocs.
L'algorithme de cette méthode est détaillé à la page 7.

- retirerGroupe()

Cette méthode supprime les blocs d'un groupe donné. N'ayant pas la position de chaque bloc du groupe, elle parcourt chaque bloc de la grille avec une boucle for et vérifie si le bloc appartient au groupe en question à l'aide d'une condition if.
Lorsqu'un bloc est trouvé, il est mis à null.

- faireTomber()

Lorsqu'un bloc est supprimé, les blocs situés au-dessus doivent tomber pour combler les vides. La méthode faireTomber permet donc d'améliorer l'expérience du jeu en faisant tomber ces blocs dans leur colonne respective.

Elle parcourt chaque colonne de bas en haut. Si elle trouve une case vide, elle cherche le premier bloc non nul situé au-dessus. Si elle en trouve un, elle fait tomber le bloc et met sa position d'origine à null.

- deplacerColonne()

La méthode deplacerColonne se charge de décaler les colonnes vides (plus un seul bloc) vers la gauche.

Une variable ColonneCible est initialisée. A l'aide d'une boucle for, l'algorithme parcourt chaque colonne de la grille et détermine si elle est vide (utilisation d'un boolean). Si le boolean n'est pas vérifié (=false), la colonne non vide est déplacée dans la ColonneCible déclarée plus tôt.

c) Classe Score

La classe Score permet d'afficher et de calculer le score du joueur à chaque coup effectuer en fonction de la taille du groupe sélectionné .
Il est composé plusieurs méthode :

- Score()

La méthode initialise le score à 0 en début de partie avant le calcul des points.

- ajouterPoints(int tailleGroupe)

La méthode s'occupe de calculer les points en fonction de la taille du groupe supprimé. Il ne peut le calculer que si la taille du groupe est de minimum 2 et le calcul avec la formule $(n - 2)^2$.

- reset()

Cette méthode est simple, elle permet au score de se réinitialiser à 0 a chaque nouvelle partie.

- getScore()

Elle permet au joueur de voir son score actuel en temps réel.

- toString()

La méthode renvoie une représentation textuelle du score qui s'affiche en au de l'écran quand on joue.

d) Classes Jeu et Main

La **classe Jeu** gère l'interface graphique du jeu. Elle hérite de JFrame et contient tous les éléments visuels et interactifs pour faire fonctionner le jeu.

Parmi les fonctionnalités de la classe Jeu on trouve :

- L'affichage des blocs sous formes d'icônes
- Gestion des évènements utilisateurs via des écouteurs d'évènements (MouseListener, ActionListener)

- ➢ Lorsque l'utilisateur clique sur un bloc, le programme détecte le groupe correspondant, le supprime, fait tomber les blocs restants, déplace les colonnes, et met à jour l'affichage
- ❖ Affichage et Mise à jour du score

La classe Jeu contient une méthode pour rafraîchir l'affichage de la grille après chaque action.

- afficherGrille()

Cette méthode génère l'affichage graphique de la grille. Elle gère notamment les événements utilisateurs lorsque la souris survole et clic sur un groupe de blocs.

Elle nettoie l'affichage précédent, et parcourt chaque case la grille via une boucle for. L'objectif étant de créer un bouton (JButton) par bloc.
D'autre part, des écouteurs d'événements ont été implémentés pour permettre de réagir aux actions du joueur.
Une réutilisation des méthodes définies précédemment dans la classe Bloc a été nécessaire.

La **classe Main** est la classe principale, c'est elle qui démarre le jeu en créant une instance de la classe Jeu.

- main()

Cette classe contient uniquement une méthode main qui sert à initialiser la fenêtre du jeu et à lancer l'interface graphique.

3. Algorithme des blocs

L'algorithme de détection des groupes de blocs est un élément fondamental au bon fonctionnement du jeu, car il permet des blocs adjacents de même typ pouvant être supprimés ensemble.
Pour ce faire, les méthodes detecterGroupes et detecterGroupe ont été définies dans la classe Bloc.

Nous avons mis en place une approche récursive pour diviser le problème en sous-problèmes. Une fonction récursive s'appelle elle-même jusqu'à atteindre un cas où l'exécution s'arrête.

- ➔ La première méthode parcourt l'ensemble de la grille de type *Bloc[][]*. Elle utilise deux boucles for imbriquées pour parcourir chaque position de la grille.

Un tableau boolean mémorise les cases déjà analysées pour éviter de traiter plusieurs fois les mêmes blocs.
Si le bloc à cet endroit n'a pas encore été visité et n'est pas nul :

- Elle crée une liste vide pour stocker les blocs connectés.
- Elle appelle la fonction récursive (méthode detecterGroupe) pour former un groupe à partir de ce bloc.
- Si la taille du groupe formé est strictement supérieur à 1, le groupe est ajouté dans la liste.

➔ La deuxième méthode est la partie récursive de l'algorithme. Elle se charge d'explorer les 4 voisins d'un bloc donné et de vérifier si ils peuvent appartenir au même groupe.
Pour cela, elle vérifie plusieurs conditions :

- Le bloc doit être dans la grille
- Le bloc ne doit pas être nul
- Le bloc ne doit pas être déjà visité
- Le bloc doit être du même type que le bloc d'origine

Si les conditions sont respectées, le bloc est marqué comme visité et est ajouté dans la liste de groupes de blocs. La méthode s'appelle alors sur ses 4 voisins directs.

Si les conditions ne sont pas respectées, l'algorithme s'arrête.

III. Conclusion

1. Conclusion Wede

La réalisation de ce projet m'a permis de mettre en pratique de nombreuses notions apprises en cours, notamment la manipulation de tableaux en deux dimensions. J'ai particulièrement apprécié travailler sur l'interface graphique, car cela m'a permis de visualiser concrètement le fonctionnement du jeu et de rendre l'expérience plus interactive pour l'utilisateur.

Le travaille en groupe sur ce projet a été tout aussi formateur. Nous avons dû apprendre à organiser notre travail, à nous répartir les tâches efficacement, et à communiquer régulièrement pour s'assurer que tout le monde avance dans la même direction. Cela m'a permis de développer mon sens de la communication mais aussi à proposer des idées tout en respectant celles du groupe.

Ce projet m'a donné envie d'aller plus loin en Java notamment en ajoutant des fonctionnalités comme la sauvegarde des parties, un classement des scores, ou encore de nouveaux éléments graphiques. Je suis fier du résultat obtenu et de la progression réalisée tout au long du développement.

2. Conclusion Shirel

D'une part, ce projet m'a permis de mieux comprendre des principes fondamentaux de la programmation orientée objet, notamment le principe de séparation des responsabilités.
Chaque classe doit avoir une seule responsabilité bien définie, et son code doit être isolé dans un fichier à part.
Créer des fichiers différents pour chaque classe assure une meilleure lisibilité et a facilité la maintenance du programme. En effet, lorsqu'une fonctionnalité a été oubliée ou qu'un bug a été détecté, il a été facile de déterminer quel fichier était responsable et de corriger l'erreur.

D'autre part, ce projet m'a permis de me familiariser davantage avec des aspects techniques de la programmation.
En effet, les notions d'héritage, d'interface et de récursivité ont été largement abordées durant le développement du jeu.
J'ai également appris à créer un fichier Makefile et à utiliser le serveur git de l'IUT.

Pour conclure, le jeu que nous avons développé a renforcé mes connaissances techniques et semble répondre aux consignes initiales mais pourrait néanmoins être amélioré en ajoutant de nouvelles fonctionnalitées :

- Un affichage "Game Over" pourrait être ajouté lorsque la grille n'est pas vide et qu'il ne reste plus aucun groupe de blocs.

- Le bouton réinitialiser devrait s'accompagner d'un bouton "aléatoire" et d'un bouton "générer à partir d'un fichier" pour que la méthode chargerDepuisFichier de la classe Grille soit utilisée.